

RELATÓRIO CONFIDENCIAL DE AUDITORIA

Relatório de Auditoria de Segurança Mimic Backup Systems

Auditoria de código-fonte cobrindo isolamento de acesso (RBAC), autorização no backend, IDOR, exposição de segredos e tratamento de entradas (XSS), com mapeamento de cada categoria para a stack real do projeto.

DATA	VERSÃO AUDITADA	REPOSITÓRIO
31 de agosto de 2026	v0.8.1	mimic_backup

2 achados de severidade baixa

3 achados informativos

0 críticos / 0 altos

Escopo auditado: Código-fonte completo do backend Go (cmd/mimic, internal/handlers, internal/middleware, internal/access, internal/models, internal/services/{ssh,sftp,alert,scheduler}, pkg/crypto), todos os templates Go HTML server-side (templates/**), arquivos de deploy (Dockerfile, docker-compose.yml, .env.example), documentação de instalação (TUTORIAL.md, DOCKER.md) e histórico completo do git (git log --all -p) em busca de segredos commitados.

Stack detectada e nota metodológica

Como cada categoria da auditoria foi mapeada para a stack real do projeto antes da varredura.

Stack técnica identificada

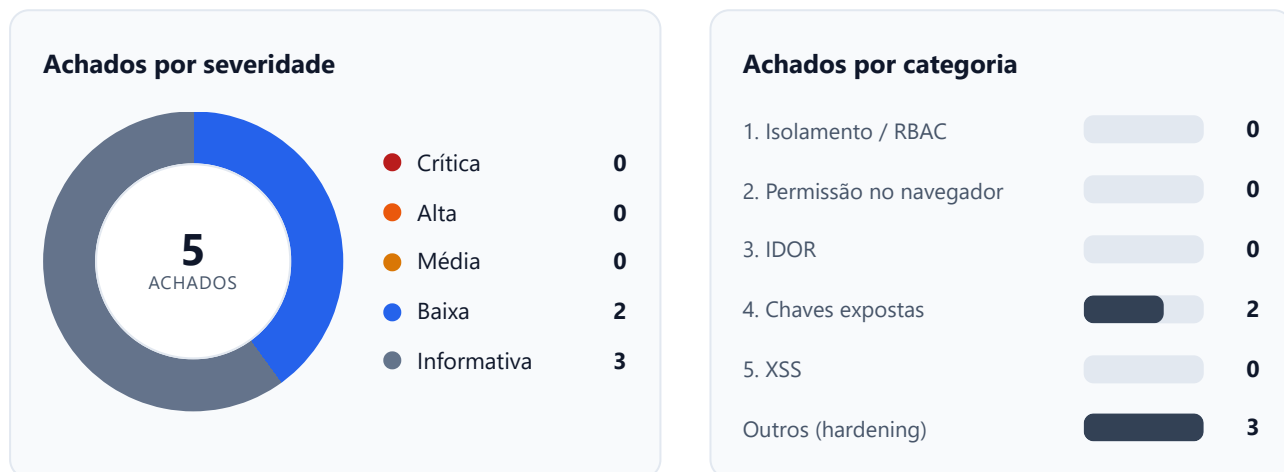
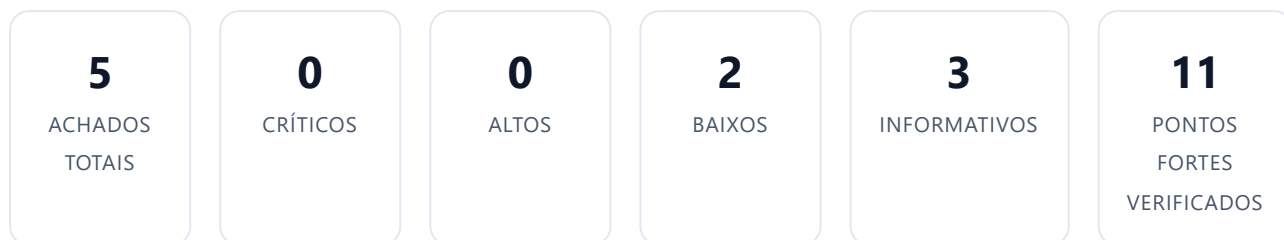
Linguagem / Runtime	Go 1.25
Framework web	Fiber v2 (gofiber/fiber)
ORM / Banco	GORM + PostgreSQL 15 (queries parametrizadas)
Autenticação	Sessão via cookie (gofiber/session), bcrypt para senhas
Autorização	RBAC por papel (Administrator/Operator/Auditor/Viewer) via middleware Fiber
Frontend	Go html/template (server-side, auto-escape) + HTMX + Alpine.js
Criptografia de segredos	AES-GCM 256-bit (pkg/crypto) para credenciais SSH/SFTP/Webhook
Deploy	Docker multi-stage + docker-compose (Postgres + app), usuário não-root

Mapeamento categoria → mecanismo real

1. Banco sem tranca (isolamento de inquilino)	O projeto não usa Supabase/RLS e não possui modelo multi-tenant (não há OrgID/TenantID/WorkspaceID em nenhuma entidade GORM). É uma aplicação single-tenant: um deployment atende uma única organização. O mecanismo de isolamento real é o RBAC (papéis Administrator/Operator/Auditor/Viewer) aplicado via middleware Fiber (RequirePermission/RequireAdmin) em cada rota. A auditoria verificou a presença desse middleware em todas as 47 rotas registradas em cmd/mimic/main.go.
2. Permissão definida no navegador	Cada elemento de UI condicionado por papel nos templates Go (CanManageUsers, CanManageNodes, comparações {{if eq .Role "..."}}) foi cruzado com o middleware da rota correspondente que o botão/link aciona em cmd/mimic/main.go.
3. IDOR	Todo handler que recebe um :id via c.Params/c.Query/FormValue (nodes, backups, users, credentials, routines, alert rules) foi lido individualmente em internal/handlers/handlers.go e internal/handlers/forms.go (arquivos completos, não amostras) para confirmar existência de verificação de posse/permissão antes de ler, alterar ou excluir o recurso.
4. Chaves expostas	Busca por regex de segredos (api key, secret key, senha literal, tokens, blocos BEGIN PRIVATE KEY) em todos os arquivos rastreados pelo git (git grep) e em todo o histórico de commits (git log --all -p), além de leitura manual de pkg/crypto/crypto.go, docker-compose.yml, Dockerfile, .env.example e TUTORIAL.md/DOCKER.md.
5. Inputs sem tratamento (XSS)	Go html/template escapa por padrão qualquer valor interpolado em contexto HTML/atributo/URL/JS. A auditoria buscou (grep) por template.HTML, funções

'safe'/'noescape' e concatenação manual de HTML nos handlers. Como não há SPA (React/Vue), buscas por innerHTML/v-html/dangerouslySetInnerHTML foram feitas nos templates HTMX/Alpine.js para confirmar que hx-swap=innerHTML sempre recebe fragmentos já renderizados (e escapados) pelo próprio servidor, não dados brutos via JS.

Resumo executivo



Nenhuma falha crítica, alta ou média foi identificada. Os cinco achados reportados são de severidade baixa ou informativa e, em sua maioria, exigem que o ator já possua privilégio elevado (Administrator) ou dependam de uma escolha de configuração de deploy/documentação — não representam uma via de comprometimento remoto não autenticado.

Pontos fortes (protegido, com evidência)

Comportamentos corretos verificados no código, listados para demonstrar a cobertura da auditoria.

● RBAC aplicado no servidor em 100% das rotas sensíveis

cmd/mimic/main.go:213-272 — todas as 29 rotas de mutação/gestão (nodes, users, credentials, routines, sftp, alerts, export, trigger de backup) passam por middleware.RequirePermission(...) ou middleware.RequireAdmin() antes do handler. Nenhuma rota de escrita foi encontrada sem gate.

● Nenhuma inconsistência entre UI condicional e backend

templates/node_list.html:11, templates/partials/node_table_body.html:85,145,153 escondem 'New Node/Edit/Delete/Manual Backup' para papéis sem permissão; as rotas correspondentes (/nodes/new, /nodes/:id/edit, /nodes/:id/delete, /nodes/:id/trigger) exigem exatamente access.ManageNodes ou access.RunBackups no servidor (cmd/mimic/main.go:214-225,283-296).

● Nenhum IDOR encontrado nas rotas CRUD

Todos os handlers com :id (NodeDetails, EditNode/SaveNode/DeleteNode, EditUser/SaveUser/DeleteUser, EditCredential/SaveCredential/DeleteCredential, EditRoutine/SaveRoutine/DeleteRoutine, EditAlertRule/SaveAlertRule/DeleteAlertRule) buscam o recurso por ID e retornam 404 se ausente; como o app é single-tenant e RBAC-gated, não há escopo de 'dono' a violar. SaveProfile (internal/handlers/forms.go:1428) deriva o ID exclusivamente de c.Locals("user_id") da sessão, nunca de parâmetro externo — impossível editar o perfil de outro usuário por esse endpoint.

● Proteções de conta e sessão bem implementadas

internal/handlers/forms.go:762-776,826-837 impedem auto-rebaixamento e remoção do último administrador; internal/handlers/auth.go:21-55 usa hash bcrypt dummy para tempo constante (anti-enumeração de usuário); sess.Regenerate() é chamado em todo login bem-sucedido (auth.go:61, setup.go:188) prevenindo session fixation.

● Nenhum segredo hardcoded no código-fonte rastreado ou no histórico git

git grep e git log --all -p não encontraram chaves de API, senhas literais ou blocos de chave privada em nenhum arquivo rastreado. .env (com segredos reais gerados localmente) está listado em .gitignore:2-3 e nunca foi commitado (git log --all --oneline -- .env retornou vazio). docker-compose.yml:11,13,34 usa a sintaxe \${VAR:?mensagem}, que interrompe a subida do container se POSTGRES_PASSWORD/SECRET_KEY não forem definidos — não há fallback inseguro.

● SECRET_KEY validado no startup, nunca aceito com menos de 32 caracteres

pkg/crypto/crypto.go:32-39 rejeita SECRET_KEY com menos de 32 caracteres; cmd/mimic/main.go:54-56 chama apccrypto.ValidateSecretKey() antes de qualquer rota subir, com log.Fatalf em caso de falha.

● Nenhuma XSS encontrada — escaping consistente

Nenhuma ocorrência de template.HTML, template.JS ou funções 'safe'/'noescape' em todo o repositório (grep). A única construção manual de HTML fora do template engine (internal/handlers/handlers.go:688, mensagem de erro do explorador SFTP) usa html.EscapeString antes de interpolar. Exportação CSV usa csvSafe() (internal/handlers/handlers.go:241-251) para neutralizar injeção de fórmula em Excel/Sheets nos campos controlados pelo usuário.

● SSRF parcialmente mitigado em webhooks de alerta

internal/services/alert/alert.go:88-120 resolve o host do webhook e bloqueia IPs loopback, privados, link-local e não especificados antes de qualquer disparo (ValidateWebhookURL é chamado em SaveAlertRule, TestAlertRule e novamente dentro de SendWebhook). Ver achado F1 para o gap residual de DNS rebinding.

- **CSRF mitigado por checagem de Origin/Referer com fail-closed**

internal/middleware/security.go:14-51 rejeita qualquer requisição de mutação (não GET/HEAD/OPTIONS) sem Origin nem Referer, e rejeita Origin/Referer que não batam com o Host — coberto por teste (internal/middleware/security_test.go). Cookie de sessão usa SameSite=Strict e HttpOnly (cmd/mimic/main.go:108-113).

- **Comandos SSH por vendor são strings estáticas, sem interpolação de dados do usuário**

internal/services/ssh/vendors/{cisco,mikrotik,huawei,juniper}.go retornam comandos fixos ("show running-config", "/export", etc.); internal/services/ssh/service.go:223-237 apenas concatena essas strings estáticas com '; ', sem incluir Node.Name/IP/Username do usuário no comando remoto — não há injeção de comando no dispositivo de rede.

- **Upload de avatar restrito e sem travessia de diretório**

internal/handlers/forms.go:1336-1404 valida tamanho (2 MB), sniffs o Content-Type real (image/jpeg ou image/png apenas) e as dimensões da imagem antes de salvar; o nome do arquivo é gerado no servidor (user- <id>-<timestamp>.ext), nunca a partir de input do usuário, e a remoção usa filepath.Base() sobre o valor armazenado — sem risco de path traversal.

Achados detalhados por categoria

SEVERIDADE	ARQUIVO:LINHA	DESCRIÇÃO
BAIXA	internal/services/alert/alert.go:88-159	SSRF via DNS rebinding (TOCTOU) na validação de webhooks de alerta
BAIXA	cmd/mimic/main.go / .env.example:main.go:108-113	Cookie de sessão sem a flag Secure por padrão
INFORMATIVA	TUTORIAL.md:63-66	Exemplo de senha literal fraca na documentação de instalação bare-metal
INFORMATIVA	pkg/crypto/crypto.go:32-61	Fallback silencioso: chave de criptografia gerada e persistida em disco se SECRET_KEY não for definida
INFORMATIVA	cmd/mimic/main.go:194-203	Rate limiting de login aplicado apenas por IP, sem bloqueio por conta

Detalhamento técnico

BAIXA**F1 SSRF via DNS rebinding (TOCTOU) na validação de webhooks de alerta****Arquivo:** `internal/services/alert/alert.go`**Linhas:** 88-159**Categoria:** Outros (SSRF)

`ValidateWebhookURL/isBlockedWebhookHost` resolvem o hostname do webhook e bloqueiam IPs privados/loopback/link-local no momento da validação (linhas 88-120). Porém `SendWebhook` (linhas 130-159) reutiliza um `http.Client` padrão que refaz a resolução DNS no momento da conexão TCP, sem fixar (pin) o IP validado. Um atacante que já possua a permissão `ManageSystem` (necessária para criar/testar Alert Rules) pode registrar um domínio com TTL curto que resolve para um IP público na validação e para 127.0.0.1/rede interna milissegundos depois, no momento real da requisição HTTP — contornando o bloqueio de SSRF.

```
func SendWebhook(url string, message string) error {
    if err := ValidateWebhookURL(url); err != nil {
        return err
    }
    ...
    client := &http.Client{Timeout: 10 * time.Second}
    resp, err := client.Do(req) // nova resolução DNS aqui, sem IP pinning
```

POR QUE É EXPLORÁVEL

Técnica clássica de bypass de SSRF (DNS rebinding): validação e uso ocorrem em dois momentos distintos, cada um com sua própria resolução de nome.

CONDIÇÕES DE EXPLORABILIDADE

Pré-condição: exige um usuário já autenticado com papel Administrator (única role com permissão `access.ManageSystem` — ver `internal/access/access.go:31-47`), controle de um domínio próprio com TTL de DNS baixo, e uma rede interna alcançável a partir do host da aplicação. Como esse ator já possui controle total do sistema, o impacto adicional é limitado a reconhecimento/pivô de rede interna a partir do servidor da aplicação.

RECOMENDAÇÃO

Resolver o host uma única vez, validar o IP resultante e reutilizá-lo na conexão real (ex.: `net.Dialer` com `DialContext` customizado que conecta diretamente ao IP validado, mantendo o header `Host` original), em vez de validar uma URL e deixar o `net/http` padrão re-resolver o DNS.

BAIXA

F2 Cookie de sessão sem a flag Secure por padrão

Arquivo: cmd/mimic/main.go / .env.example

Linhas: main.go:108-113; .env.example:8

Categoria: Outros (Config)

O Secure flag do cookie de sessão só é ativado se a variável de ambiente `COOKIE_SECURE` estiver explicitamente definida como 'true'. O `.env.example` traz `COOKIE_SECURE=false` como valor padrão sugerido, e o `docker-compose.yml` usa `$(COOKIE_SECURE:-false)` como fallback. Uma instalação seguindo a documentação sem alterar esse valor roda com cookies de sessão transmissíveis em texto claro caso a aplicação seja exposta via HTTP (ex.: atrás de um reverse proxy mal configurado, ou em rede interna sem TLS).

```
store := session.New(session.Config{
    Expiration:    24 * time.Hour,
    CookieHTTPOnly: true,
    CookieSameSite: "Strict",
    CookieSecure:  strings.EqualFold(os.Getenv("COOKIE_SECURE"), "true"),
})
```

POR QUE É EXPLORÁVEL

Sem o atributo Secure, o navegador pode enviar o cookie de sessão em uma conexão HTTP não criptografada caso exista qualquer caminho de rede sem TLS até a aplicação, permitindo captura de sessão por um atacante na mesma rede (ex.: Wi-Fi compartilhado, proxy interno).

CONDIÇÕES DE EXPLORABILIDADE

Só é explorável se a instância for servida via HTTP (sem TLS) em algum ponto do caminho de rede do cliente — não afeta instalações atrás de TLS/reverse proxy com HTTPS ponta a ponta. O próprio README recomenda 'Enable this when the application is served through HTTPS', mas o valor padrão do exemplo é 'false'.

RECOMENDAÇÃO

Inverter o padrão para 'true' e documentar a exceção explícita para ambientes de desenvolvimento local sem TLS, ou detectar automaticamente X-Forwarded-Proto quando atrás de um proxy confiável.

INFORMATIVA

F3 Exemplo de senha literal fraca na documentação de instalação bare-metal

Arquivo: TUTORIAL.md

Linhas: 63-66, 83

Categoria: 4. Chaves expostas

O tutorial de instalação manual mostra a criação do usuário PostgreSQL com a senha literal 'password' e a mesma string na `DATABASE_URL` do serviço `systemd` de exemplo. Há um aviso ('> Warning: Change `password` to a secure password.') logo acima, mas o valor em si não é um placeholder obviamente inválido — instaladores apressados podem copiar e colar sem trocar.

```
> **Warning:** Change `password` to a secure password.
```sql
CREATE USER mimic WITH ENCRYPTED PASSWORD 'password';
...
...
Environment="DATABASE_URL=postgres://mimic:password@localhost:5432/mimic_db?sslmode=disable"
```

## POR QUE É EXPLORÁVEL

Não é uma vulnerabilidade de código, mas um risco operacional: se o operador ignorar o aviso, o banco fica protegido por uma senha trivialmente adivinhável.

## CONDIÇÕES DE EXPLORABILIDADE

Requer que o operador siga o tutorial sem seguir a instrução de troca. E que o PostgreSQL esteja acessível pela rede (por padrão `TUTORIAL.md` configura o serviço para `localhost`).

## RECOMENDAÇÃO

Substituir 'password' por um placeholder que falha de forma óbvia se não for trocado, ex. '<TROQUE\_ESTA\_SENHA>', em vez de uma senha que 'funciona' se deixada como está.

## INFORMATIVA

F4

**Fallback silencioso: chave de criptografia gerada e persistida em disco se SECRET\_KEY não for definida****Arquivo:** pkg/crypto/crypto.go**Linhas:** 32-61**Categoria:** 4. Chaves expostas

Quando SECRET\_KEY não está no ambiente, loadSecretKey() tenta ler .mimic\_secret e, se ausente, gera 24 bytes aleatórios, codifica em base64 e grava em .mimic\_secret com permissão 0600. Isso evita uma chave hardcoded (positivo), mas é uma degradação silenciosa: a aplicação sobe normalmente sem avisar que está operando com uma chave não gerenciada pelo operador. Em Docker (docker-compose.yml) isso é mitigado porque SECRET\_KEY é obrigatória via \${SECRET\_KEY:?...}, mas uma instalação bare-metal (TUTORIAL.md) sem exportar SECRET\_KEY cairia nesse fallback sem qualquer log de alerta.

```
secretFile := ".mimic_secret"
data, err := os.ReadFile(secretFile)
if err == nil { ... }
// Generate new key
newKey := make([]byte, 24)
...
os.WriteFile(secretFile, []byte(b64Key), 0600)
```

**POR QUE É EXPLORÁVEL**

Se o arquivo .mimic\_secret for perdido (ex.: reinstalação, migração de host sem copiar o arquivo), todas as credenciais SSH/SFTP/Webhook já criptografadas no banco tornam-se permanentemente indecifráveis — mais um risco de continuidade operacional do que de confidencialidade direta, mas vale nota de segurança porque mascara uma configuração ausente.

**CONDIÇÕES DE EXPLORABILIDADE**

Não é explorável remotamente; é um problema de gestão de configuração/chaves em instalações que não seguem o Docker Compose (que já força a variável).

**RECOMENDAÇÃO**

Emitir um log.Printf de aviso claro quando o fallback for usado ('SECRET\_KEY not set, generated and persisted a new key at .mimic\_secret — back this file up'), e considerar recusar a subida em modo produção (ex. quando um APP\_ENV=production estiver definido).

## INFORMATIVA

## F5 Rate limiting de login aplicado apenas por IP, sem bloqueio por conta

Arquivo: cmd/mimic/main.go

Linhas: 194-203

Categoria: Outros (Auth)

O limiter de /login permite 10 tentativas por minuto por IP (chave padrão do middleware limiter do Fiber é c.IP()). Não existe bloqueio complementar por username após N falhas. Um atacante distribuído (múltiplos IPs/proxies) pode tentar senhas contra uma única conta sem esbarrar nesse limite, já que cada IP tem sua própria cota.

```
loginLimiter := limiter.New(limiter.Config{
 Max: 10,
 Expiration: time.Minute,
 ...
})
app.Post("/login", loginLimiter, authHandler.PostLogin)
```

## POR QUE É EXPLORÁVEL

Rate limiting só por IP não impede brute force distribuído contra uma conta específica; bcrypt (DefaultCost) e a checagem de tempo constante (auth.go) já elevam bastante o custo por tentativa, mas o limite de taxa é a defesa que faltaria numa distribuição de origem.

## CONDIÇÕES DE EXPLORABILIDADE

Requer um atacante com acesso a múltiplos endereços IP (botnet, proxies rotativos) e um username válido conhecido ou adivinhável.

## RECOMENDAÇÃO

Adicionar um contador de falhas por username (ex. Redis/DB) com bloqueio temporário progressivo após N tentativas, complementando o limite por IP já existente.

## Recomendações priorizadas

---

**P1**

Corrigir o SSRF por DNS rebinding em `internal/services/alert/alert.go`: resolver o host uma única vez, validar o IP e reutilizá-lo na conexão HTTP real (Dialer customizado), em vez de validar a URL e deixar o `net/http` padrão re-resolver o DNS no momento da requisição.

---

**P2**

Inverter o padrão de `COOKIE_SECURE` para `'true'` em `.env.example` e `docker-compose.yml`, com instrução explícita de exceção apenas para desenvolvimento local sem TLS.

---

**P3**

Higienizar exemplos de credenciais na documentação (`TUTORIAL.md`) trocando `'password'` por um placeholder que falha visivelmente se não for substituído, e emitir um log de aviso quando `pkg/crypto` usar o fallback de geração automática de `SECRET_KEY` em disco.

---

**P4**

Reforçar o rate limiting de `/login` com um contador de falhas por username (além do limite por IP já existente), para mitigar brute force distribuído contra uma conta específica.

---

# Issues para o GitHub

Texto completo em Markdown, pronto para copiar e colar em uma nova issue.

--- ISSUE 1 ---

# [Segurança] SSRF via DNS rebinding na validação de webhooks de alerta

**Labels sugeridas:** security, baixa

## Descrição do problema

ValidateWebhookURL valida o host do webhook e bloqueia IPs privados/loopback no momento do salvamento/teste da regra, mas SendWebhook deixa o net/http padrão re-resolver o DNS no momento da conexão real, sem fixar o IP validado. Um domínio com TTL curto pode resolver para um IP público na validação e para um alvo interno (127.0.0.1, rede privada) no momento do disparo real, contornando o bloqueio de SSRF. Requer papel Administrator (permissão ManageSystem) para cadastrar a regra de alerta.

## Evidência

**internal/services/alert/alert.go:130-159**

```
...
func SendWebhook(url string, message string) error {
 if err := ValidateWebhookURL(url); err != nil {
 return err
 }
 ...
 client := &http.Client{Timeout: 10 * time.Second}
 resp, err := client.Do(req)
 ...
}
```

## Impacto

Um Administrator malicioso ou uma conta de Administrator comprometida pode usar o servidor da aplicação como pivô para escanear/alcançar serviços internos que não deveriam ser expostos, contornando o bloqueio de SSRF já existente.

## Sugestão de correção

Resolver o hostname uma única vez, validar o(s) IP(s) resultante(s) e reutilizá-los na conexão TCP real (por exemplo, um http.Transport com DialContext customizado que conecta diretamente ao IP validado, preservando o header Host original), em vez de validar a URL textual e deixar o cliente HTTP padrão resolver o DNS de novo no momento do request.

## Critérios de aceite

- [ ] SendWebhook (e qualquer outro chamador de rede a partir de URL fornecida pelo usuário) conecta ao IP resolvido e validado, não a um novo lookup de DNS
- [ ] Teste automatizado cobrindo um cenário de DNS rebinding (mock de resolver retornando IPs diferentes em validação vs. conexão) confirma o bloqueio
- [ ] Comportamento de bloqueio para IPs privados/loopback/link-local continua funcionando (regressão)

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

# [Segurança] Cookie de sessão sem flag Secure por padrão

**\*\*Labels sugeridas:\*\*** security, baixa

**## Descrição do problema**

CookieSecure só é habilitado se `COOKIE_SECURE=true` for definido explicitamente. `.env.example` e `docker-compose.yml` usam `'false'` como padrão sugerido, então uma instalação seguindo a documentação sem alterar esse valor roda com cookies de sessão sem o atributo Secure, que poderiam ser transmitidos em texto claro se a aplicação for exposta via HTTP em algum trecho do caminho de rede.

**## Evidência**

**\*\*cmd/mimic/main.go:108-113\*\***

...

```
store := session.New(session.Config{
 Expiration: 24 * time.Hour,
 CookieHTTPOnly: true,
 CookieSameSite: "Strict",
 CookieSecure: strings.EqualFold(os.Getenv("COOKIE_SECURE"), "true"),
})
...
```

**\*\*env.example:7-8\*\***

...

# Enable this when the application is served through HTTPS.

`COOKIE_SECURE=false`

...

**## Impacto**

Em uma instalação exposta via HTTP (sem TLS ponta a ponta), o cookie de sessão pode ser capturado por um atacante na mesma rede (ex. Wi-Fi compartilhado, proxy interno mal configurado), permitindo sequestro de sessão.

**## Sugestão de correção**

Inverter o valor padrão sugerido para `'true'` em `.env.example` e `docker-compose.yml`, deixando clara a exceção apenas para desenvolvimento local sem TLS; opcionalmente, detectar automaticamente X-Forwarded-Proto quando atrás de um proxy confiável.

**## Critérios de aceite**

- [ ] `.env.example` e `docker-compose.yml` passam a sugerir `COOKIE_SECURE=true` por padrão
- [ ] `README/DOCKER.md` documentam explicitamente quando é seguro definir `'false'` (apenas dev local)

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

# [Segurança] Hardening de segredos: exemplo de senha fraca na documentação e fallback silencioso de SECRET\_KEY

**Labels sugeridas:** security, informativa

## ## Descrição do problema

Dois achados relacionados a gestão de segredos, agrupados por serem do mesmo tema: (1) TUTORIAL.md usa a senha literal 'password' em um exemplo de criação de usuário PostgreSQL e na DATABASE\_URL de um serviço systemd – com aviso ao lado, mas sem ser um placeholder obviamente inválido. (2) pkg/crypto/crypto.go gera e persiste silenciosamente uma nova SECRET\_KEY em .mimic\_secret quando a variável de ambiente não está definida, sem log de aviso, o que pode mascarar uma configuração ausente em instalações fora do Docker Compose (que já torna a variável obrigatória).

## ## Evidência

**TUTORIAL.md:63-66, 83**

...

> **Warning:** Change `password` to a secure password.

```
CREATE USER mimic WITH ENCRYPTED PASSWORD 'password';
```

...

```
Environment="DATABASE_URL=postgres://mimic:password@localhost:5432/mimic_db?sslmode=disable"
```

...

**pkg/crypto/crypto.go:32-61**

...

```
secretFile := ".mimic_secret"
```

```
data, err := os.ReadFile(secretFile)
```

```
if err == nil { ... }
```

```
// Generate new key
```

```
newKey := make([]byte, 24)
```

...

```
os.WriteFile(secretFile, []byte(b64Key), 0600)
```

...

## ## Impacto

(1) Banco de dados protegido por senha trivial se o operador ignorar o aviso. (2) Perda do arquivo .mimic\_secret (reinstalação, migração sem backup) torna credenciais SSH/SFTP/Webhook já criptografadas permanentemente indecifráveis, sem que o operador tenha sido avisado de que dependia desse arquivo.

## ## Sugestão de correção

Trocar 'password' por um placeholder que falha visivelmente se não for substituído (ex. '<TROQUE\_ESTA\_SENHA>').

Emitir log.Printf de aviso quando o fallback de geração de SECRET\_KEY for usado, e considerar recusar a subida quando um modo de produção estiver sinalizado.

## ## Critérios de aceite

- [ ] TUTORIAL.md não contém mais uma senha literal 'funcional' em exemplos de configuração
- [ ] Aplicação loga um aviso claro no startup quando SECRET\_KEY é gerada automaticamente
- [ ] Documentação menciona explicitamente a necessidade de backup do .mimic\_secret quando esse fallback é usado

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

# [Segurança] Rate limiting de login apenas por IP, sem bloqueio por conta

**\*\*Labels sugeridas:\*\*** security, informativa**## Descrição do problema**

O limiter de POST /login usa a chave padrão do middleware Fiber (endereço IP do cliente), permitindo 10 tentativas por minuto por IP. Não existe um contador complementar por username, então um atacante com múltiplos IPs (proxies, botnet) pode tentar senhas contra uma conta específica sem esbarrar em limite algum.

**## Evidência****\*\*cmd/mimic/main.go:194-203\*\***

...

```
loginLimiter := limiter.New(limiter.Config{
 Max: 10,
 Expiration: time.Minute,
 LimitReached: func(c *fiber.Ctx) error { ... },
})
app.Post("/login", loginLimiter, authHandler.PostLogin)
...
```

**## Impacto**

Brute force distribuído contra uma conta específica não é mitigado pelo rate limit atual, embora o custo por tentativa já seja elevado por bcrypt + comparação de tempo constante.

**## Sugestão de correção**

Adicionar um contador de falhas de login por username (Redis, tabela dedicada ou coluna em User) com bloqueio temporário progressivo após N tentativas, complementando – não substituindo – o rate limit por IP já existente.

**## Critérios de aceite**

- [ ] Login com um mesmo username falhando N vezes seguidas (mesmo de IPs diferentes) resulta em bloqueio temporário da conta
- [ ] O bloqueio por conta não introduz um vetor de enumeração de usuários (mensagem de erro permanece genérica)

--- FIM ISSUE 4 ---